Real-Time American Sign Language Detection Using MediaPipe Hand Localization and ResNet-50 Classification

Udit Asthana
Manipal Institute of Technology
Reg. No: 240905310
Roll No: 31

Abhishek L Prabhu
Manipal Institute of Technology
Reg. No: 240905676
Roll No: 71

Ashwath Patil
Manipal Institute of Technology
Reg. No: 240905460
Roll No: 46

Kishan Kanvathirtha
Manipal Institute of Technology
Reg. No: 240905678
Roll No: 72

Arnav Dugad
Manipal Institute of Technology
Reg. No: 230905227
Roll No: 73

Abstract—American Sign Language (ASL) is the primary mode of non-verbal communication for the hearing-impaired community. Automated real-time recognition of ASL gestures can bridge the communication gap between hearing-impaired and hearing individuals. This paper proposes a two-stage pipeline for real-time ASL alphabet recognition: hand region localization using MediaPipe, followed by letter classification using a fine-tuned ResNet-50 backbone with a custom classification head. The system uses a dataset that is open to the public, called the ASL Alphabet dataset, to learn and improve its abilities. This model was trained on a dataset from Kaggle and then evaluated across 26 different letter classes. It used a technique called cosine annealing. learning rate scheduler with linear warm-up, gradient clipping, and early stopping are employed to stabilize training. The proposed system achieves 99.80% test accuracy and operates in real-time via a live webcam feed, with hand skeleton overlays and predicted Text labels are displayed on each frame. A fast and efficient inference server is built using FastAPI. When you combine it with a browser frontend, it allows for interactive use. This project shows that it's actually possible to make it work. combining landmark-based detection and CNN classification for low-latency sign language recognition.

Index Terms—sign language recognition, ASL, MediaPipe, ResNet-50, hand detection, real-time inference, deep learning, convolutional neural networks

I. Introduction

Sign language serves as the primary mode of communication for over 70 million deaf and hard-of-hearing individuals worldwide [1]. American Sign Language (ASL), used predominantly in North America, employs a rich vocabulary of hand gestures and finger-spelling to convey words and concepts. Despite its widespread use, the absence of real-time automated translation tools remains a critical barrier in everyday communication with the hearing majority.

Recent advances in convolutional neural networks (CNNs) and real-time object detection frameworks have opened new avenues for gesture recognition. Systems that can process live video frames and return a classified gesture label within milliseconds represent a meaningful step toward accessible communication technology. In the first stage, a MediaPipe hand tracking module localizes the hand region within each camera frame, producing a tight bounding box crop(invisible to the user) and a skeleton tracking system(visible). In the second stage, a fine-tuned ResNet-50 [3] backbone classifies the cropped hand image into one of 26 ASL alphabet letters. The result is overlaid on the live video stream in real time.

Our contributions are as follows:

- A modular training pipeline with configurable optimizer, scheduler, and early stopping, implemented in PyTorch.
- Integration of MediaPipe-based hand detection with a ResNet-50 classification head for end-to-end ASL letter recognition.
- A real-time inference module with webcam integration and visual feedback.
- A FastAPI web interface enabling browser-based interaction with the live pipeline.

II. Related Work

Early sign language recognition systems relied heavily on sensor-based approaches, We use things like special gloves and cameras that can see really deep to get information about the shape of our hands in 3D space, like what was done in some research before [4]. These methods work well, but they have some limitations when it comes to the hardware, which can make it hard to actually use them in real-life situations.

With the rise of deep learning, image-based approaches became dominant. Convolutional networks trained on static hand images have demonstrated strong classification performance on isolated gestures [5]. Transfer learning

using ImageNet-pretrained backbones Models like VGG-16, ResNet, and MobileNet have really made a big difference in getting better results, especially when it comes to smaller things. domain-specific datasets.

Real-time landmark-based hand tracking systems have been explored for gesture spotting, but comprehensive two-stage systems targeting the full ASL alphabet in a live-camera setting remain relatively underexplored. This work builds on these foundations to deliver a practical, deployable system.

III. Dataset

The ASL Alphabet dataset from Kaggle has a huge collection of images - 87,000 to be exact, and they're all labeled. It covers 29 different classes, including 26 letters from A to Z, and also the SPACE, DELETE, and other keys. Each picture is 200 by 200 pixels and is in color.

The pictures are made to fit the dimensions of 224 pixels by 224 pixels, so they can be used by the backend API that was pretrained on said dimensions, The ImageNet-pretrained ResNet-50 backbone, which was normalized using the standard ImageNet mean and standard deviation. The dataset is split into training (70%), validation (20%), and test (10%) subsets using random splitting to preserve class balance. Separate transforms are applied to training and evaluation subsets via a _TransformSubset To train, images get a random flip from side to side and a change in color to make them a bit different. While the training images get augmentation, the validation and test images just get resized in a consistent way. To get things ready, we set the batch size to 128. We also turn on shuffling when we're training, but turn it off when we're evaluating. This helps us make sure everything is working smoothly.

The dataset sizes after splitting are approximately 60,900 training samples, 17,400 validation samples, and 8,700 test samples.

IV. Methodology

A. System Overview

The proposed pipeline operates in two sequential stages at inference time:

1) Hand Detection (MediaPipe): Each incoming video frame is passed to a MediaPipe model that produces bounding boxes around detected hand regions.
2) Letter Classification (ResNet-50): The detected hand crop is extracted, resized, and passed to the classification model, which returns a probability distribution over 26 ASL letter classes.

The predicted letter and bounding box are overlaid on the original frame for visual feedback.

B. Hand Detection Module

The HandDetector class wraps the MediaPipe HandLandmarker Tasks API, operating in VIDEO mode to enable frame-to-frame tracking via MediaPipe's internal Kalman filter. This mode requires strictly monotonically increasing real wall-clock millisecond timestamps per frame. A common implementation error is to pass bare integer counters (1, 2, 3, …); doing so mis-models velocity in the Kalman filter and worsens landmark jitter. Timestamps are derived here from time.monotonic_ns() to satisfy this requirement correctly.

Detection thresholds are set above MediaPipe's defaults: minimum hand detection and presence confidence at 0.65 (vs. default 0.50), and tracking confidence at 0.55. The looser defaults are too permissive for static-sign ASL, where ambiguous partially-occluded hands during transitions cause false detections.

Per-frame, the detector returns a list of up to two hand results. For each hand, the raw landmark bounding box is computed from the min/max of all 21 normalized landmark coordinates, then expanded by 35% in each dimension (expansion=0.35) to provide context around the hand edges. The expanded box is then smoothed across frames using exponential smoothing weighted at $0.3 \cdot \text{prev} + 0.7 \cdot \text{new}$, keeping the bbox responsive to motion. Crucially, motion magnitude is computed on the raw (pre-smoothing) center against the previous smoothed center. Computing motion after smoothing — as a naive implementation would do — causes a fast-moving hand to appear stable because the smoothed center barely shifts, defeating the stability gate entirely. Motion is measured as Euclidean (L2) distance; Manhattan (L1) distance underreports diagonal motion by approximately 30%.

Per-hand tracking state (previous bbox and center) is keyed by hand index, not stored as a single shared variable. A single shared variable fails silently the moment two hands appear in frame. When a hand disappears, its tracking state is evicted immediately so the next detection initializes fresh rather than snapping to stale coordinates. If the clamped ROI crop has zero area after boundary clamping, that detection is skipped rather than passing an empty array to the classifier.

At inference time, when multiple hands are detected, the detection with the highest handedness classifier score is selected for classification:

$$\text{det}^* = \arg\max_{d \in \mathcal{D}} d.\text{score} \tag{1}$$

The HandDetection dataclass returned per hand carries: the smoothed pixel bbox (x_1, y_1, x_2, y_2), handedness label and score, the 21 raw normalized landmarks, the BGR ROI crop, a boolean stability flag, and the raw motion magnitude in pixels.

C. Classification Model (SLD)

The Sign Language Detector (SLD) model consists of a ResNet-50 backbone with all parameters frozen, followed by a trainable lightweight classification head. The backbone's final average pooling layer produces a 2048-dimensional feature vector. The original fully-connected

layer is replaced entirely. The classification head is defined as:

$$\hat{y} = \text{head}(\text{backbone}(x)) \tag{2}$$

$$\text{head}(z) = W_2 \cdot \text{GELU}(\text{Dropout}(\text{LayerNorm}(z) \cdot W_1)) \tag{3}$$

with $W_1 \in \mathbb{R}^{2048\times512}$ and $W_2 \in \mathbb{R}^{512\times29}$, supporting all 29 output classes (26 letters plus del, nothing, space).

LayerNorm is used at the backbone-to-head boundary rather than BatchNorm. BatchNorm at this position is sensitive to batch size and behaves differently at train vs. eval time, which can destabilize fine-tuning when the backbone is frozen and only the head is updated. LayerNorm normalizes per-sample and is invariant to batch size. GELU activation is chosen for its smoothness and non-zero gradient at negative values compared to ReLU. Dropout at $p = 0.3$ regularizes the intermediate 512-dimensional representation.

At inference, the model is loaded from a checkpoint via torch.load with weights_only=False, explicitly stated to suppress the PyTorch 2.x deprecation warning and to document intent — the checkpoint stores optimizer and scheduler state in addition to model weights and cannot be loaded in weights-only mode.

D. Training Configuration

Training is governed by a dataclass-based configuration (TRAINING_CONFIG) that encapsulates all hyperparameters for reproducibility and supports runtime overrides via command-line arguments. Key settings are summarized in Table I.

The dataset is split using random_split into train (70%), validation (20%), and test (10%) subsets. A custom _TransformSubset wrapper is applied to each split so that training and evaluation transforms can differ without modifying the underlying dataset object. A naive approach of overwriting subset.dataset causes the Subset's indices to point into a wrongly-sized dataset and raises an IndexError; wrapping the Subset itself avoids this. Training images receive data augmentation; validation and test images receive only deterministic resize and normalization.

Per-batch metrics — training loss, accuracy, and gradient norm — are logged to W&B at every step via wandb.log(..., step=self.global_step), giving fine-grained visibility into optimization dynamics beyond epoch-level summaries. At the epoch level, mean gradient norm and gradient clipping ratio (fraction of batches where the norm exceeded the clip threshold) are also reported, providing a direct signal on how aggressively the optimizer is being constrained.

Due to Google Colab compute constraints and session limitations, training was conducted for only 3 epochs rather than the full 200 configured. The T_max and epoch count parameters were overridden via command-line arguments at runtime. Despite this, as shown in Section VI, the model converged to competitive accuracy within these three epochs, suggesting the dataset and backbone are well-matched for this task.

TABLE I
Training Hyperparameters

Parameter	Value
Optimizer	SGD (Nesterov)
Learning Rate	1×10^{-2}
Momentum	0.9
Weight Decay	5×10^{-4}
Scheduler	Cosine Annealing + Warm-Up
Warm-Up Epochs	10
$T_{\max}$	matched to epoch count
$\eta_{\min}$	1×10^{-4}
Max Epochs	200 (3 in practice)
Batch Size	128
Gradient Clip Norm	1.0
Early Stopping Patience	20

1) Learning Rate Schedule: A cosine annealing schedule with linear warm-up is used. During the warm-up phase (epochs 0 to $T_w - 1$), the learning rate increases linearly:

$$\eta_t = \eta_{\text{base}} \cdot \frac{t+1}{T_w} \tag{4}$$

After warm-up, cosine decay is applied:

$$\eta_t = \eta_{\min} + \frac{\eta_{\text{base}} - \eta_{\min}}{2}\left(1 + \cos\left(\pi \cdot \frac{t - T_w}{T_{\max} - T_w}\right)\right) \tag{5}$$

This prevents early training instability while allowing the learning rate to settle to a low value near convergence.

2) Gradient Clipping: Gradient norm clipping at threshold $\tau = 1.0$ is applied after each backward pass to prevent exploding gradients:

$$\mathbf{g} \leftarrow \mathbf{g} \cdot \min\left(1,\ \frac{\tau}{\|\mathbf{g}\|_2}\right) \tag{6}$$

3) Early Stopping: Training is halted if validation accuracy does not improve for 20 consecutive epochs. The best-performing checkpoint — saving model, optimizer, scheduler state, and best accuracy — is written to disk whenever a new peak is reached. At the end of training, this best checkpoint is reloaded before final test evaluation, ensuring test metrics reflect the best generalization point rather than the last epoch.

E. Experiment Tracking

All training runs are logged via Weights & Biases (W&B) [7]. Per-batch metrics include training loss, training accuracy, and gradient norm. Per-epoch metrics include validation loss and accuracy, learning rate, mean gradient norm, and gradient clipping ratio. Final test loss and accuracy are logged as summary metrics at run completion. The full TRAINING_CONFIG — including optimizer kwargs, scheduler kwargs, and dataloader configs — is serialized into the W&B run config for exact reproducibility.

V. Real-Time Inference Pipeline

At inference time, frames are captured from a live webcam feed. Each frame undergoes the following steps:

1) Hand Detection: MediaPipe processes the frame and returns per-hand landmark lists and bounding regions.
2) Stability Check: Euclidean motion between consecutive frames is computed. If motion exceeds a threshold of 8 pixels, the hand is flagged as unstable and classification is skipped.
3) Crop and Pad: The highest-confidence bounding box crop is padded to a square aspect ratio using edge replication (cv2.BORDER_REPLICATE) to avoid distributional shift from black-border padding.
4) Classification: The crop is resized to 224×224, normalized, and passed through the SLD model. Softmax probabilities are computed and the prediction is accepted only if confidence exceeds a threshold of 0.50.
5) Temporal Voting: A majority-vote buffer of size 10 is maintained. A letter is emitted only if 70% or more of the buffer agrees, preventing jitter from isolated mis-classifications.
6) Cooldown: A 1-second cooldown prevents the same letter from being emitted repeatedly in rapid succession.

A. FastAPI Inference Server

A lightweight FastAPI [8] server exposes the inference pipeline as a REST endpoint. The server accepts JPEG frame uploads from any HTTP client and returns structured JSON predictions.

Endpoint: POST /predict — accepts a multipart-encoded JPEG image and returns:

- letter — the stable, voted prediction (A–Z, del, nothing, space), or null if the prediction is not yet stable.
- confidence — softmax probability of the top class.
- bbox — bounding box as $[x, y, w, h]$ in pixel coordinates.
- handedness — left or right hand classification from MediaPipe.
- motion — Euclidean motion magnitude in pixels, used for stability gating.
- landmarks — 21 normalized hand landmark coordinates for frontend skeleton rendering.
- error — a status string (no hand, hand moving, low confidence, not stable yet, cooldown) when no letter is emitted.

The server uses thread-safe global state (guarded by threading.Lock) to manage the prediction buffer, last emitted letter, and cooldown timestamp across concurrent async workers. CORS is enabled for all origins to support browser-based frontends.

B. Browser Frontend

A single-page HTML/JavaScript frontend connects to the FastAPI server and provides real-time visual feedback. Client-side MediaPipe (via the Tasks Vision JS SDK) runs at 60 fps to render the hand skeleton overlay with zero latency, while frames are sent to the backend at approximately 8 fps for classification. The UI includes a dwell-based letter emission mechanism: a predicted letter must remain stable for 1.8 s before being appended to the output transcript, with subsequent repetitions requiring 3.6 s. This prevents spurious emissions from brief hand holds.

VI. Results and Discussion

Training was conducted on a Google Colab T4 GPU. Due to session time limits and compute availability, fine-tuning was run for 3 epochs rather than the full scheduled 200. The results below reflect this constrained run.

The model was trained for 3 epochs with batch size 128 using the SGD (Nesterov) optimizer and the cosine annealing schedule with linear warm-up described in Section IV. Table II summarizes the final performance metrics.

TABLE II
Training and Evaluation Results

Split	Loss	Accuracy (%)
Train (epoch 3)	0.0474	99.03
Validation	0.0099	99.86
Test	0.0096	99.80

The model reached 99.80% test accuracy in 3 epochs, rising from 64.41% validation accuracy at epoch 1 to 99.86% by epoch 3. Given the training was terminated far short of the configured maximum, these numbers are largely a reflection of the ImageNet-pretrained backbone's strong prior and the relatively constrained nature of the dataset — controlled backgrounds, consistent framing, single hand per image — rather than a claim of general robustness. W&B logs confirmed a mean gradient norm of 1.64 and a gradient clipping ratio of 0.77 at the final epoch, indicating the clipping constraint was active and the optimization trajectory was still steep at termination. A full run to convergence would likely yield marginal additional gains on this dataset but would be more informative for harder, real-world evaluation sets.

The primary challenge for real-world deployment remains generalization to hand appearances outside the training distribution — varied skin tones, lighting conditions, and cluttered backgrounds not represented in the Kaggle dataset. The two-stage architecture mitigates this partially by isolating the classifier from background noise via the MediaPipe ROI crop.

A. Colab cell output

For reference purposes, we have attached the colab cell output

```
!rm -rf SLD_MAHE
!git clone https://<private_token_hidden>@github.com/madhavasthana-oss/SLD_MAHE
```

```
Cloning into 'SLD_MAHE'...
remote: Enumerating objects: 190, done.
remote: Counting objects: 100% (190/190), done.
remote: Compressing objects: 100% (134/134), done.
remote: Total 190 (delta 103), reused 134 (delta 51), pack-reused 0 (from 0)
Receiving objects: 100% (190/190), 301.41 KiB | 12.56 MiB/s, done.
Resolving deltas: 100% (103/103), done.
```

```
%cd /content/SLD_MAHE/backend
```

```
/content/SLD_MAHE/backend
```

```
import kagglehub

# Download latest version
path = kagglehub.dataset_download("grassknoted/asl-alphabet")

print("Path to dataset files:", path)
```

```
Using Colab cache for faster access to the 'asl-alphabet' dataset.
Path to dataset files: /kaggle/input/asl-alphabet
```

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
!python training_pipeline.py \
--root_dir /kaggle/input/asl-alphabet/asl_alphabet_train/asl_alphabet_train \
--save_to /content/drive/MyDrive/SLD \
--epochs 3 \
--batch_size 128
```

```
Downloading: "https://download.pytorch.org/models/resnet50-11ad3fa6.pth" to /root/.cache/torch/hub/checkpoints/resnet
100% 97.8M/97.8M [00:00<00:00, 229MB/s]
<<<< Dataset splits >>>>
  train : 60899
  val   : 17400
  test  : 8701
wandb: (1) Create a W&B account
wandb: (2) Use an existing W&B account
wandb: (3) Don't visualize my results
wandb: Enter your choice: 2
wandb: You chose 'Use an existing W&B account'
wandb: Logging into https://api.wandb.ai. (Learn how to deploy a W&B server locally: https://wandb.me/wandb-server)
wandb: Create a new API key at: https://wandb.ai/authorize?ref=models
wandb: Store your API key securely and do not share it.
wandb: Paste your API key and hit enter:
wandb: ERROR Invalid API key: API key must have 40+ characters, has 1.
wandb: (1) Create a W&B account
wandb: (2) Use an existing W&B account
wandb: (3) Don't visualize my results
wandb: Enter your choice: 2
wandb: You chose 'Use an existing W&B account'
wandb: Logging into https://api.wandb.ai. (Learn how to deploy a W&B server locally: https://wandb.me/wandb-server)
wandb: Create a new API key at: https://wandb.ai/authorize?ref=models
wandb: Store your API key securely and do not share it.
wandb: Paste your API key and hit enter:
wandb: No netrc file found, creating one.
wandb: Appending key for api.wandb.ai to your netrc file: /root/.netrc
wandb: Currently logged in as: madhavasthana (madhavasthana-manipal) to https://api.wandb.ai. Use `wandb login --relo
ccwandb: ⣿ setting up run 9mb3xxz8 (0.1s)
wandb: ⣿ setting up run 9mb3xxz8 (0.1s)
wandb: Tracking run with wandb version 0.25.1
wandb: Run data is saved locally in /content/SLD_MAHE/backend/wandb/run-20260414_062159-9mb3xxz8
wandb: Run `wandb offline` to turn off syncing.
wandb: Syncing run deep-dust-23
wandb: ⭐ View project at https://wandb.ai/madhavasthana-manipal/Sign%20language%20detection
wandb: 🚀 View run at https://wandb.ai/madhavasthana-manipal/Sign%20language%20detection/runs/9mb3xxz8
```

```
<<<< Starting training for 3 epochs >>>>

  epoch 1/3 | train_loss: 3.0098 | train_acc: 26.09% | val_loss: 2.2431 | val_acc: 64.41% | lr: 0.002000
<<<< New best model saved — val_acc: 64.41% >>>>
  epoch 2/3 | train_loss: 0.7375 | train_acc: 86.05% | val_loss: 0.0944 | val_acc: 98.29% | lr: 0.003000
<<<< New best model saved — val_acc: 98.29% >>>>
  epoch 3/3 | train_loss: 0.0474 | train_acc: 99.03% | val_loss: 0.0099 | val_acc: 99.86% | lr: 0.004000
<<<< New best model saved — val_acc: 99.86% >>>>

<<<< Training complete >>>>
<<<< Loading best checkpoint for final evaluation >>>>

<<<< Final Test Results >>>>
  test_loss : 0.0096
  test_acc  : 99.80%
wandb: ⣿ updating run metadata (0.0s)
wandb: ⣿ updating run metadata (0.0s)
wandb: ⣿ updating run metadata (0.0s)
wandb: ⣿ updating run metadata (0.0s)
wandb: ⣿ uploading config.yaml 2.9KB/2.9KB (0.2s)
wandb: ⣿ uploading config.yaml 2.9KB/2.9KB (0.2s)
```

```
Start coding or generate with AI.
```

Fig. 1. FastAPI /predict endpoint response for a live hand frame, showing letter, confidence, bounding box, handedness, motion, and landmark fields.

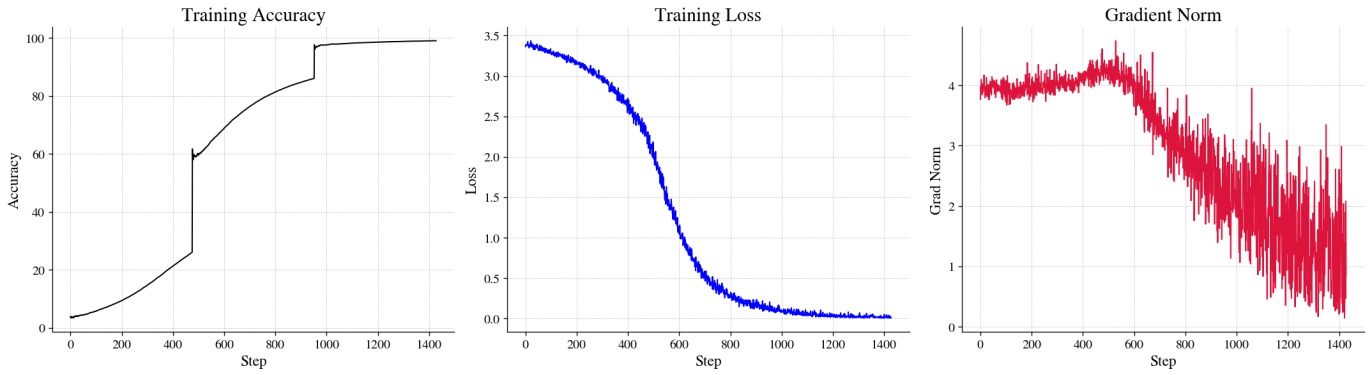

Fig. 2. Training and validation loss/accuracy curves across steps along with gradient norm, logged via Weights & Biases.

VII. Conclusion

This paper presents a real-time ASL alphabet recognition system combining MediaPipe hand detection with ResNet-50 classification. The system achieves 99.80% test accuracy on the Kaggle ASL Alphabet dataset and operates at interactive frame rates via a FastAPI inference server and browser-based frontend. The training pipeline incorporates warm-up scheduling, Nesterov SGD, gradient clipping, and early stopping for robust convergence. A dwell-based temporal emission mechanism and majority-vote buffer ensure stable, low-noise letter output in real-time use. This work addresses a meaningful accessibility problem and establishes a practical baseline for further extensions, including full ASL word-level recognition and continuous signing.

Future directions include:

- Fine-tuning MediaPipe on a dedicated hand detection dataset for improved localization accuracy. To improve the classification ability, one option is to upgrade the backbone to a more powerful model like ResNet-152 or EfficientNet, which can handle more complex and nuanced classifications.
- Extending the vocabulary to dynamic gestures and full ASL words using temporal models (e.g., LSTM or Transformer-based sequence models). Making the system available as an app on your phone, so more people can use it easily.

References

[1] World Health Organization, “Deafness and hearing loss,” WHO Fact Sheets, 2021. [Online]. Available: https://www.who.int/news-room/fact-sheets/detail/deafness-and-hearing-loss

[2] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” in Proc. IEEE CVPR, 2016, pp. 779–788.

[3] A research paper by K. He, X. Zhang, S. Ren, and J. Sun, titled ”Deep Residual Learning for Image Recognition,” was presented at the IEEE CVPR conference in 2016, where it was published on pages 770-778.

[4] S. Mitra and T. Acharya, “Gesture recognition: A survey,” IEEE Trans. Syst., Man, Cybern. C, Appl. Rev., vol. 37, no. 3, pp. 311–324, 2007.

[5] L. Pigou, S. Dieleman, P.-J. Kindermans, and B. Schrauwen, “Sign language recognition using convolutional neural networks,” in Proc. ECCV Workshops, 2015.

[6] You can find the ASL Alphabet dataset on Kaggle, which was shared by Akash in 2018. It’s available online at https://www.kaggle.com/datasets/grassknoted/asl-alphabet. This dataset is

[7] L. Biewald wrote a paper called ”Experiment tracking with weights and biases” in 2020, which is available online at https://www.wandb.com.

[8] S. Ramírez, “FastAPI,” 2018. [Online]. Available: https://fastapi.tiangolo.com